Section 07: Human-in-the-Loop

Agents and Humans Collaborating Through Graph Interrupts

LangGraph v1.0.x (March 2026)

Mental Model: Why Human-in-the-Loop?

The Big Picture

- Agents make mistakes or take risky actions
- Humans provide **oversight**, **judgment**, **final approval**
- Interrupt graph, show human state, wait for decision
- Resume when human approves or edits

Essential for Production

HITL is critical for high-stakes decisions (financial, medical, legal).

Breakpoints: The Two Interrupt Types

interrupt_before

- Pauses **before** a node executes
- Human reviews state and inputs, then approves
- Example: block before calling "delete_database" tool

interrupt_after

- Pauses **after** a node executes
- Human sees output, can edit before next node
- Example: review agent's decision before proceeding

Setting Breakpoints: interrupt_before

Configure Interrupts at Compile Time

```
graph = builder.compile(
  checkpointer=checkpointer,
  interrupt_before=["dangerous_tool_node"]
)
result = graph.invoke(
  {"messages": [HumanMessage("Delete old files")]},
  config={"configurable": {"thread_id": "123"}}
)
```

Breakpoint List

Pass list of node names. Graph pauses before executing those nodes.

Setting Breakpoints: interrupt_after

Pause After Node Execution

```
graph = builder.compile(
  checkpointer=checkpointer,
  interrupt_after=["agent_node"]
)
result = graph.invoke(
  {"messages": [HumanMessage("Write email")]},
  config={"configurable": {"thread_id": "123"}}
)
```

Use Cases

Review draft outputs, approve decisions, edit state before proceeding.

How Interrupts Work

Interrupt Flow

```
config = {"configurable": {"thread_id": "conv_123"}}
try:
  result = graph.invoke(
    {"messages": [HumanMessage("Action")]},
    config=config
  )
except GraphInterrupt:
  state = graph.get_state(config)
  print(f"Paused at: {state.next}")
  print(f"State: {state.values}")
```

Async-Friendly

Interrupts work with both sync and async execution.

Resuming After Interrupt

Resume with None Input

```
config = {"configurable": {"thread_id": "conv_123"}}
state = graph.get_state(config)
print(f"Pending node: {state.next}")
human_approval = input("Approve? (y/n): ")
if human_approval == "y":
  result = graph.invoke(None, config=config)
  print(result)
```

Resume with None

`invoke(None, config=...)` continues from last checkpoint without new input.

Approval Patterns: Pre-Tool Execution

Approve Tool Calls

```
graph = builder.compile(
  checkpointer=checkpointer,
  interrupt_before=["tools"]
)
result = graph.invoke(input, config)
state = graph.get_state(config)
tool_calls = state.values["messages"][-1].tool_calls
for call in tool_calls:
  approved = human_approves(call)
  if not approved:
    graph.update_state(config, {"skip_tool": True})
result = graph.invoke(None, config)
```

Safety First

Block dangerous tool calls before execution.

Editing State Before Resuming

Manual State Modification

```
config = {"configurable": {"thread_id": "conv_123"}}
state = graph.get_state(config)
print(f"Current messages: {state.values['messages']}")
edited_messages = state.values["messages"]
edited_messages.append(
  HumanMessage("Correction: use this instead")
)
graph.update_state(config, {"messages": edited_messages})
result = graph.invoke(None, config)
```

Human Feedback Loop

Let humans correct mistakes in graph state mid-execution.

The Command Primitive

Command Types

- `Command(resume=value)`: pass human input to graph
- `Command(goto=node_name)`: jump to specific node
- `Command(resume=None)`: continue with no new input

Advanced Control

Commands enable precise control over graph execution flow.

Using Command(resume=value)

Pass Human Input Back

```
from langgraph.types import Command
state = graph.get_state(config)
human_decision = input("Approve? Yes/No: ")
if human_decision == "Yes":
  graph.update_state(
    config,
    Command(resume=HumanMessage("Approved"))
  )
result = graph.invoke(None, config)
```

User Feedback Integration

Commands let you inject human decisions back into graph state.

Using interrupt() Inside Nodes

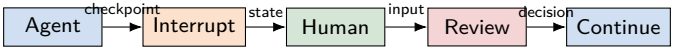
Inline Interrupts

```python
from langgraph.types import interrupt
def approval_node(state):
  decision = interrupt(
    f"Approve action: {state['action']}?"
  )
  if decision == "approve":
    return {"approved": True}
  else:
    return {"approved": False}
builder.add_node("approval", approval_node)
```

Dynamic Interrupts

Use `interrupt()` to pause mid-node, not just at node boundaries.

TikZ Diagram: HITL Flow



Full Loop

Agent pauses, human reviews, returns decision. Loop continues or stops.

Multiple Breakpoints in One Graph

Multi-Point Interrupts

```
graph = builder.compile(
  checkpointer=checkpointer,
  interrupt_before=[
    "approve_decision",
    "execute_payment"
  ],
  interrupt_after=["draft_email"]
)
```

Flexible HITL

Different nodes can have different approval strategies.

Dynamic Approval: Only High-Risk Actions

Conditional Approval

- Don't interrupt for every tool call
- Only approve high-risk: delete, transfer money, send emails
- Low-risk tools (search, read) execute automatically

User Experience

Minimize interruptions. Only pause when humans genuinely need to decide.

Dynamic Approval: Implementation

Risk-Based Routing

```
def should_interrupt(state):
  last_tool = state["messages"][-1].tool_calls[0]
  if last_tool["name"] in ["delete", "transfer", "send"]:
    return "approval_node"
  else:
    return "execute_tools"
builder.add_conditional_edges(
  "agent", should_interrupt,
  {"approval_node": "approval_node", ...}
)
```

Smart Filtering

Use conditional edges to route risky vs safe tools.

Timeout and Fallback Patterns

Timeout Strategy

- If human doesn't respond in N seconds, what happens?
- Option 1: Continue with default (risky)
- Option 2: Fail gracefully (safe)
- Option 3: Escalate to manager

Design Decision

Default depends on your risk tolerance and business logic.

Timeout Implementation

Async Timeout with asyncio

```
import asyncio
async def wait_for_approval(state, timeout=30):
  try:
    decision = await asyncio.wait_for(
      human_approval_future,
      timeout=timeout
    )
    return {"approved": decision}
  except asyncio.TimeoutError:
    return {"approved": False, "reason": "timeout"}
```

Production Ready

Always include timeouts for interactive workflows.

Common Pitfalls

No Checkpointer with HITL

- Without checkpointer, state is lost when interrupt fires
- Solution: always use checkpointer with HITL

Lost Interrupts

- Catching exceptions without handling interrupts properly
- Solution: distinguish `GraphInterrupt` from other exceptions

Poor UX

- Too many interrupts frustrate users
- Solution: only interrupt for genuinely risky actions

Summary: HITL Cheat Sheet

Quick Reference

```
graph = builder.compile(
  checkpointer=checkpointer,
  interrupt_before=["risky_node"]
)
result = graph.invoke(input, config)
state = graph.get_state(config)
if state.next:
  human_approves = input("OK? ")
  if human_approves:
    graph.invoke(None, config)
```

Self-Check: Do You Understand?

1. What's the difference between interrupt_before and interrupt_after?
2. How do you resume after an interrupt?
3. Why is a checkpointer essential for HITL?
4. How do you edit state before resuming?
5. What are Command types and when do you use them?
6. How do you implement dynamic (risk-based) approval?
7. What's a good timeout strategy for human approval?